



Figure 1: gRPC model

gRPC and CNCF

gRPC was released by Google more than a year back and since then, there has been a lot of momentum towards getting the industry to consider this as the glue for micro-services communication, especially when latency is a key factor. Recently, it got adopted as a top level project in the Cloud Native Computing Foundation (CNCF), which is a big move towards the wider adoption of gRPC. It joined the likes of Kubernetes, Prometheus and others that are considered to be the foundation of a cloud native application.

Wire Protocol - ProtocolBuffers

ProtocolBuffers is both an interface definition language and a message format. It is primarily used to define the service interface, i.e., what the service methods are and the messages that are exchanged. The message formats are also defined via their individual parts.

This protocol definition file can then be used to generate both the server side and client side bindings, or you can dynamically load the definition and provide the implementations. ProtocolBuffers is currently available in two versions: proto2 and proto3, but we will go with the latter, which is the latest since it has support for a wide range of languages.

The best way to understand ProtocolBuffers is via an example that we will build over the course of this article. This will include the server and client implementations.

We are going to build a service called *OSFY Stats*. This service provides two methods, one for retrieving the top articles from the OSFY website and another method to retrieve the stats on a particular article, like the number of views, likes and so on. We are not going to provide a real implementation later, but this exercise is more to get an idea of how to start building the specification and message formats as per the ProtocolBuffers standard.

The proto file (*osfy_stats.proto*) is shown below:

```
syntax = "proto3";
```

```
package osfy_stats;
```

```
//Service. define the methods that the grpc server can expose to the client.
```

```
service OSFYStatsService {
    rpc toparticles (TopArticlesRequest) returns
    (TopArticlesResponse);
    rpc articleStats (ArticleStatsRequest) returns
    (ArticleStatsResponse);
}
```

```
//Message Type definition for TopArticlesRequest
message TopArticlesRequest{
    int32 numResults = 1;
}
```

```
//Message Type definition for TopArticlesResponse
message TopArticlesResponse {
    repeated Article articles = 1;
}
```

```
//Message Type definition for Article
message Article {
    int32 id = 1;
    string title = 2;
    string url = 3;
}
```

```
//Message Type definition for ArticleStatsRequest
message ArticleStatsRequest {
    int32 id = 1;
}
```

```
//Message Type definition for ArticleStatsResponse
message ArticleStatsResponse {
    int32 id = 1;
    int32 numViews = 2;
    int32 numLikes = 3;
    int32 numComments = 4;
}
```

Let us go through the proto file in brief:

- At the top of the file, we specify the version of ProtocolBuffers and that the service resides in a package named *osfy_stats*.
- We then define the service named *OSFYStatsService* and specify the two methods that it exposes, i.e., *toparticles* and *articleStats*.
- The input and output message formats are specified, and the message formats for each of the request and response messages are defined in the same file. Note that the ProtocolBuffers specification supports various scalar types (int32, float and string) and also other complex stuff like *repeatable*, *Nested* and more. The unique numbers